

ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."

— Spandan Prayas Patra

TABLE OF CONTENTS

- Chapter 1: Introduction & Philosophical Architecture
 - Chapter 2: Variables, Optional Types & Expression Syntax
 - Chapter 3: The 3-Level Native Interactivity Architecture
 - Chapter 4: Control Flow & Iteration
 - Chapter 5: Functions, Actions, Interfaces & Async / Await
 - Chapter 6: Pattern Matching & Decision Engine
 - Chapter 7: Exception Handling & Error Control
 - Chapter 8: Built-in Collections — Lists, Arrays, Dictionaries, Sets
 - Chapter 9: String Manipulations, Math & DateTime Primitives
 - Chapter 10: File I/O, System Operations & Security
 - Chapter 11: Frontend Structural Markup (`.enlgf` → HTML5)
 - Chapter 12: Styling & Design Systems (`.enlgd` → CSS3)
 - Chapter 13: Client-Side Scripting (`.enlgs` → JavaScript ES6+)
 - Chapter 14: Database Schemas & Queries (`.enlgdb` → SQL)
 - Chapter 15: Native NLP & AI Primitives
 - Chapter 16: EnLang Web Server & Multi-File Architecture
 - Chapter 17: Canonical Grammar Rules, Order & Syntax Layout Specification
 - Chapter 18: Complete Production Case Study — Lumina Workspace
 - Appendix A: Universal Natural Syntax Cheatsheet
 - Appendix B: CLI & EnLang Package Manager (EPM) Manual
-

CHAPTER 1: INTRODUCTION & PHILOSOPHICAL ARCHITECTURE

EnLang is a deterministic, multi-target compiler and runtime engine created by Spandan Prayas Patra to convert natural English into 1:1 clean, production-grade native targets: Python 3, HTML5, CSS3, JavaScript ES6+, and SQL.

CHAPTER 5: FUNCTIONS, ACTIONS, INTERFACES & ASYNC / AWAIT

EnLang provides flexible, natural syntaxes for function declarations, invocations, and return values, while remaining 100% backward compatible with standard function syntax.

5.1 Function & Action Declarations

Style A: Modern Developer Shorthand (`fn`)

```
fn add_numbers(a, b):  
    return a plus b  
  
set total to add_numbers(10, 20)  
display total
```

Style B: Signature EnLang Natural Expression (`define function / action`)

```
define function print_rec taking n:  
    if n is greater than 10 then:  
        return  
    display n  
    run print_rec with (n plus 1)  
  
run print_rec with 1
```

Style C: Action Phrase (`action <name> taking <args>:`)

```
action calculate_tax taking price, rate:  
    result is price times rate  
  
call calculate_tax with (1000, 0.18) and store in tax  
display tax
```

Style D: Legacy Standard (`function <name>(<args>):`)

```
function greet(name):  
    display "Hello, " plus name  
  
greet("Spandan")
```

5.2 Invocation / Call Keywords Summary

Functions can be invoked naturally using any of the following verbs:

- `run <name> with <args>` (e.g. `run print_rec with 1`)
- `call <name> with <args> and store in <var>` (e.g. `call calculate_tax with (1000, 0.18) and store in total`)
- `do <name> with <args>` (e.g. `do greet with "Spandan"`)
- `<name>(<args>)` (Direct call)

5.3 Return Value Keywords

- `return <expr>`
- `result is <expr>`
- `give back <expr>`

APPENDIX A: UNIVERSAL SYNTAX REFERENCE

Category	EnLang Syntax Example	Native Transpilation
Function (Fn)	<code>fn add(a, b): return a plus b</code>	<code>def add(a, b): return a + b</code>
Function (Natural)	<code>define function greet taking name:</code>	<code>def greet(name):</code>
Action	<code>action calc taking price, rate:</code>	<code>def calc(price, rate):</code>
Call & Store	<code>call calc with (100, 0.1) and store in t</code>	<code>t = calc(100, 0.1)</code>
Run Action	<code>run print_numbers with 1</code>	<code>print_numbers(1)</code>
Return Result	<code>result is a plus b / give back a plus b</code>	<code>return a + b</code>

Copyright © 2026 Spandan Prayas Patra. All rights reserved.

Published under the Open EnLang Specification License.